

TRIMAZKON

Editace v programu – Manuál



JHV-ENGINEERING s.r. o.

Nádražní 641

Pardubice - Rosice 533 51

Česká Republika

Rev. Popis

A Výchozí dokument

Datum

16. říjen 2025

Autor

Ing. Jakub Hlaváček

Obsah

1	Git lokace aplikace	1
1.1	Změna git repo názvu a git ownera – autoupdate	1
2	Příprava prostředí pro editaci programu	2
2.1	Potřebné knihovny	2
3	Modulové rozdělení aplikace	4
4	Úvod do struktury kódu aplikace	5
5	Debugging aplikace	8
6	Release nové verze aplikace (nový installer)	9
7	Úprava installeru v orca.exe	10
8	Instalace programu	12
8.1	Odinstalace programu	13
9	Licencování programu	14
9.1	Šifrování nových licencí	15
9.2	Změna kontaktního emailu pro požadavek licence	15
10	Ovládání přes systémovou nabídku „tray icons“	16
11	Seznam souborů potřebných k uchování mimo git	17

1 Git lokace aplikace

<https://github.com/Jakubhl/Work>

V readme.md je přehled funkcí, co aplikace dokáže.

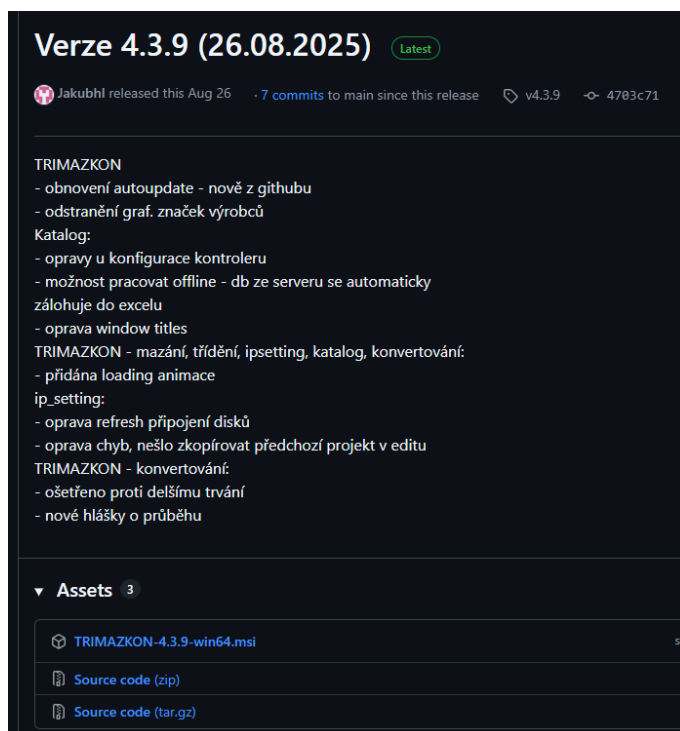
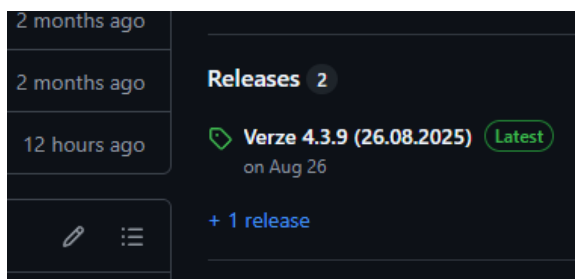
1.1 Změna git repo názvu a git ownera – autoupdate

Pokud se naklonuje repozitář na jiný git – je v aplikaci potřeba změnit název git repozitáře a git repo ownera. Je tak možné udělat přímo v kódu v horních částech GUI modulů. Nebo vždy u .json konfiguračního souboru, který si aplikace sama vygeneruje po instalaci.

```
{
  "app_settings": {
    "default_path": "C:/Users/",
    "maximalized": "ano",
    "show_changelog": "ne",
    "app_zoom": 130,
    "app_zoom_checkbox": "ano",
    "tray_icon_startup": "ne",
    "default_language": "cz",
    "tooltip_status": "ano",
    "github_repo_owner": "Jakubhl",
    "github_repo": "Work",
    "licence_email_contact": "jakub.hlavacek@jhv.cz"
  },
  "sort_conv_settings": {
    "supported_formats_sorting": [
      "bmp",

```

To vše za účelem funkčního **autoupdate** (všem, co mají nainstalovanou aplikaci se zobrazí, že je dostupná nová verze ke stažení). Majitel repozitáře vytvoří nový release s novým tagem. Ovšem pokud se změní lokace repozitáře – musí ti, co mají aplikaci již nainstalovanou přepsat konfigurační soubor (verze 4.4.0 +), viz obr. výše.



V sekci releases – draft a new release, stačí přidat do assets installer (.msi), do poznámek napsat nějaké release notes a změnit tag na číslo verze.

2 Příprava prostředí pro editaci programu

Aplikace TRIMAZKON byla vyvíjena ve vývojovém prostředí Visual studio code.
Programovací jazyk: Python, verze: 3.12

- <https://www.python.org/downloads/windows/>
- Při instalaci zaškrtnout přidat do system path
- Otevřít vývojové prostředí a zvolit si python jako interpretera

2.1 Potřebné knihovny

- Postup instalace:
 - o Otevřít terminál
 - o Příkaz: *pip install název_knihovny*

```
PS C:\Users\kubah\Desktop\JHV\Work\TRIMAZKON> pip install wmi
Collecting wmi
  Downloading WMI-1.5.1-py2.py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: pywin32 in c:\users\kubah\appdata\local\programs\python\python312\lib\site-packages (from wmi) (311)
Downloading WMI-1.5.1-py2.py3-none-any.whl (28 kB)
Installing collected packages: wmi
Successfully installed wmi-1.5.1
PS C:\Users\kubah\Desktop\JHV\Work\TRIMAZKON> █
```

Seznam knihoven:

- o customtkinter
- o pillow
- o psutil
- o cryptography
- o wmi
- o pyperclip
- o requests
- o pywin32
- o openpyxl
- o pyodbc
- o xlwings
- o cx_Freeze
- o pystray

2. možnost: pokus o export = requirements.txt, neozkoušeno. Spustit v terminalu:

pip install -r requirements.txt

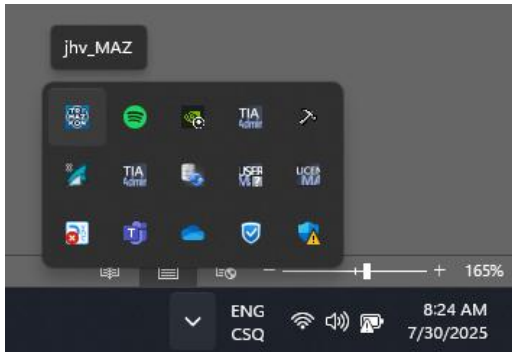
Nesmí být podtrhnutý žádný import (viz obr.) – vždy v horní části kódu:

```
import customtkinter
import os
import time
from PIL import Image, ImageTk
import Sorting_option_v5 as Trideni
import Deleting_option_v2 as Deleting
import Converting_option_v3 as Converting
import catalogue_maker_v9 as Catalogue
# import sharepoint_download as download_database
import IP_setting_v6 as IP_setting
import trimazkon_tray_v5 as trimazkon_tray
import string_database
from tkinter import filedialog
import tkinter as tk
import threading
import shutil
import ctypes
import win32pipe, pywintypes, psutil#,win32file
import subprocess
from win32api import *
from win32gui import *
import win32con
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization, hashes
import datetime
import wmi
import json
# import struct
import winreg
import pyperclip
import requests
```

3 Modulové rozdělení aplikace

Aplikace má vždy hlavní GUI soubor, ve kterém běží hlavní smyčka. Aplikace je rozdělena na 3 samostatné celky, které se volí při instalaci. Každý gui soubor si generuje vlastní .json configy.

Každý celek obsahuje .py soubor s release notes a default hodnotami, [engine služby tray](#) (viz obrázek) a další podružné soubory.



Rozdělení:

- 1) **JHV_APP_GUI_v17.py**
 - a. string_database.py
 - b. [trimazkon_tray_v5.py](#)
 - c. catalogue_maker_v9.py
 - d. db_read_catalogue_v2.py
 - e. Converting_option_v3.py
 - f. Deleting_option_v2.py
 - g. Sorting_option_v5.py
- 2) **JHV_MAZ_GUI_v4.py**
 - a. string_database_MAZ.py
 - b. [trimazkon_tray_MAZ_v3.py](#)
 - c. Deleting_option_v2.py
- 3) **IP_SET_GUI_v1.py**
 - a. ip_set_changelog.py
 - b. [ip_only_tray_v1.py](#)
 - c. IP_setting_v6.py

A další soubory jako:

- **generovani_MSI_TRIMAZKON_v2.py**
 - o slouží ke generování instaléru .msi
 - o viz další kapitoly: 6 Release nové verze aplikace (nový installer)
- **sifrovani_licence.py**
 - o slouží ke generování dlouhodobé licence pomocí zašifrovaných klíčů
 - o viz další kapitoly: 9.1 Šifrování nových licencí

4 Úvod do struktury kódu aplikace

```
1 > import sys...
5 # from psutil import process_iter as psutil_process_iter
6 # from os.path import basename as os_path_basename
7 # from os.path import exists as os_path_exists
8 # from os import getpid as os_get_pid
9
10 > class initial_tools:...
51
52     testing = False
53
54     global_recources_load_error = False
55     global_licence_load_error = False
56     exe_path = sys.executable
57     exe_name = os.path.basename(exe_path)
58     config_filename = "TRIMAZKON.json"
59     app_name = "TRIMAZKON"
60     app_version = "4.4.0"
61     github_repo_owner = "Jakubhl"
62     github_repo = "Work"
63     licence_email_contact = "jakub.hlavacek@jhv.cz"
64     loop_request = False
65     root = None
66     print("exe name: ",exe_name)
67 > if testing:...
69
70     app_running_status = initial_tools.check_runing_app_duplicity()
71     print("already opened app status (initial image?): ",app_running_status)
72     open_image_only = False
73 > if len(sys.argv) > 1 and app_running_status == True:...
78
79 > if not open_image_only:...
2089
2090 > class system_pipeline_communication: # vytvoření pipeline serveru s pipe názvy
2356
2357 > if not open_image_only:...
2466
2467 > elif open_image_only: # snaha o co nejrychlejší odeslání parametrů na server
2477
2478 > if not open_image_only:...
7952
```

Kvůli možnosti systémového otevírání TRIMAZKONu jako výchozího prohlížeče obrázků je ošetřen podmínkami „open_image_only“ aby se nenačítali všechny knihovny a všechny podružné moduly při kliknutí na obrázek.

Aplikace reaguje na tyto systémové commandy:

```
app_running_status = initial_tools.check_runing_app_duplicity()
print("already opened app status (initial image?): ",app_running_status)
open_image_only = False
if len(sys.argv) > 1 and app_running_status == True:
    used_cmd_calls = ["deleting",
                     "trigger_by_tray",
                     "run_tray",
                     "open_task_list",
                     "open_log_list",
                     "app_shutdown",
                     "edit_existing_task",
                     "settings_tray",
                     "settings_tray_del",
                     "admin_menu",
                     "installer_call",
                     "manual_ip_setting",
                     "admin_ip_setting"]
    if str(sys.argv[1]) not in used_cmd_calls:
        if sys.argv[0] != sys.argv[1]:
            open_image_only = True
```

Zejména automatické mazání – aplikace ani nemusí být spuštěna. Tato část kódu ovšem vylučuje tyto commandy a ověřuje, zda jen není otevřen obrázek (výchozí prohlížeč obr.)

Tyto třídy:

<code>class Subwindows: ...</code>	Obsahuje dialogová okna, která se různě volají v aplikaci
<code>class WindowsBalloonTip: ...</code>	Engine pro windows systémovou notifikaci, že právě proběhlo mazání souborů.
<code>class Tools: ...</code>	Tato třída obsahuje pouze classmethody – pomocné nástroje, které jsou často využívány

Třída „**system_pipeline_communication**“ zajišťuje spuštění serveru, když je aplikace zapnutá – z důvodu aby se zabránilo duplicitám běžící aplikace a za účely pouhého předání příkazů.

Současně obsahuje, co se má odesílat běžící aplikaci.

Příklad: mám na pozadí ip_setting a kliknu na obrázek – jen odešle lokaci obrázku a příkaz k otevření. Nebo ani nemusím mít otevřené okno a jen povolený běh tray na pozadí – ušetří se vždy čas při spouštění.

```

if not open_image_only:
    initial_path = Tools.get_init_path()
    print("init path: ",initial_path)
    app_icon = Tools.resource_path('images/logo_TRIMAZKON.ico')
    app_licence_validity = Tools.check_licence()
    load_gui=True

    print("SYSTEM: ",sys.argv)
    if len(sys.argv) > 1:
        > if global_licence_load_error and sys.argv[1] != "installer_call": # jen když je s
        > elif sys.argv[1] == "deleting": ...
        > elif sys.argv[1] == "run_tray": ...
        > # spuštění aplikace přes system tray nabídku
        > elif sys.argv[1] == "trigger_by_tray": ...
        > elif sys.argv[1] == "manual_ip_setting": ...
        > elif sys.argv[1] == "open_task_list": ...
        > elif sys.argv[1] == "open_log_list": ...
        > elif sys.argv[1] == "app_shutdown": ...
        > elif sys.argv[1] == "edit_existing_task": ...
        > elif sys.argv[1] == "settings_tray" or sys.argv[1] == "settings_tray_del" or sys.

    #Musí být až tady, protože musu terminatenout aplikaci (vyše v kodu)
    app_running_status = initial_tools.check_runing_app_duplicity()
    print("already opened app status: ",app_running_status)

```

V této části se naslouchá systémovým vstupům/ commandům a rozhoduje se, jestli se bude načítat GUI nebo jen přijímat informace k odeslání na běžící server.

```

if not open_image_only:
    class Tooltip: ...

    class main_menu: ...

    class Image_browser: # Umožňuje procházet obrázky a přitom například vybrané přesouvat do jiné složky ...

    class Advanced_option: # Umožňuje nastavit základní parametry, které ukládá do textového souboru ...

    class Converting_option: # Spouští možnosti konvertování typu souborů ...

    class Deleting_option: # Umožňuje mazat soubory podle nastavených specifikací ...

    class Sorting_option: # Umožňuje nastavit možnosti třídění souborů ...

    class IP_manager: # Umožňuje nastavit možnosti třídění souborů ...

    class Catalogue_maker: # Umožňuje nastavit možnosti třídění souborů ...

    if load_gui:
        if not app_running_status:
            menu = main_menu(root)
            menu.menu(initial=True)

    def start_new_root(): ...

    if loop_request:
        root.mainloop()

```

Následuje hlavní rozdělení.

Tooltip je engine pro zobrazování detailních informací po přejetí kurzorem nad tlačítky.

Main_menu je hlavní menu, spouští se vždy defaultně.

Image_browser = gui prohlížeče obrázků.

Advanced_option = gui sekce nastavení parametrů aplikace

Converting_option = gui, které umožňuje konvertovat soubory .ifz na .bmp nebo .jpg. využívá externí aplikaci v adresáři “convert_application”

Deleting_option = gui pro mazání a nastavení aut. Mazání

Sorting_option = gui pro různé možnosti třídění souborů

IP_manager = gui pro nastavování ip adres a mapování disků.

Catalogue_maker = gui pro vytváření katalogů hw komponentů – možnost čerpat hw komponenty z eplan SQL databáze nebo z exelu. – možnosti importu .xml, exportu: .xml, .xlsx, .xlsm a do SQL.

Zmíněné třídy si volají externí .py soubory, které obsahují hlavní funkcionalitu.

load_gui proměnná – otevři menu pokud je shozený gui, ale pořád běží hlavní smyčka s rootem (třeba když běží tray ikona).

start_new_root() = vytvoř nový root a spust’ smyčku

loop_request proměnná = spust’ jen smyčku

5 Debugging aplikace

Většina .py souborů má v horní části kódu globální proměnnou „testing“. Zejména GUI moduly.

```
24 # import struct
25 import winreg
26 import pyperclip
27 import requests
28
29 testing = False
30
31
32 global_recources_load_error = False
33 global_licence_load_error = False
```

Stačí přepsat tento bool na True, ovšem pozor! Před zabalením aplikace je potřeba překontrolovat, že je všude testing proměnná False.

Je to proto, jelikož aplikace se z části řídí podle PID procesů windows a komunikuje přes pipeline server v reálném čase – proto se v debuggeru nespustí správně, protože není .exe

- K ověření před exportem aplikace slouží v generovani_MSI_TRIMAZKON_v2.py tato část kódu:

```
def check_dependencies():
    import JHV_APP_GUI_v17 as main_gui
    import JHV_MAZ_GUI_v4 as jhv_maz_gui
    import IP_SET_GUI_v1 as ip_set_gui

    if not (main_gui.app_version == jhv_maz_gui.trimazkon_version == ip_set_gui.trimazkon_version):
        raise ValueError("Version mismatch between applications")
    if main_gui.testing == False and ip_set_gui.testing == False and jhv_maz_gui.testing == False:
        raise ValueError("Some app has enabled testing mode")
    if int(main_gui.app_version.replace(".", "")) < int(new_version.replace(".", "")):
        raise ValueError("Version mismatch")

new_version = "4.3.9"
# check_dependencies()
```

Před generování znovu zakomentovat.

Pozn.: Při vypnutí hlavního okna aplikace běží stále na pozadí tray aplikace, pokud není nastaveno jinak.

6 Release nové verze aplikace (nový installer)

```
testing = True

global_recources_load_error = False
global_licence_load_error = False
exe_path = sys.executable
exe_name = os.path.basename(exe_path)
config_filename = "TRIMAZKON.json"
app_name = "TRIMAZKON"
app_version = "4.3.9"
github_repo_owner = "Jakubhl"
github_repo = "Work"
licence_email_contact = "jakub.hlavacek@jhv.cz"
loop_request = False
root = None
print("exe name: ", exe_name)
if testing:
    exe_name = "trimazkon_test.exe"
```

- 1) Je potřeba nastavit globální proměnnou `app_version` ve **všech gui souborech** na vyšší
- 2) Nastavi verzi v souboru generovani `MSI_TRIMAZKON_v2.py`:

```
new_version = "4.3.9"
# check_dependencies()
```

- a.
- 3) Spustit tento soubor přes cmd console (v cestě se souborem) s tímto příkazem:
 - a. `python generovani_MSI_TRIMAZKON_v2.py bdist_msi`
 - b. nastane delší generování (cx_Freeze) +- 2 min
 - c. vzniknou dva adresáře:

build	16/10/2025 22:25	File folder
convert_application	12/09/2025 09:23	File folder
del_licence	12/09/2025 09:23	File folder
dist	12/09/2025 09:23	File folder

- d.
- e. Build adresář se může hned celý smazat
- f. Dist adresář obsahuje installer:

TRIMAZKON-4.4.0-win64.msi	16/10/2025 22:25	Windows Installer ...	33,932 KB
---------------------------	------------------	-----------------------	-----------

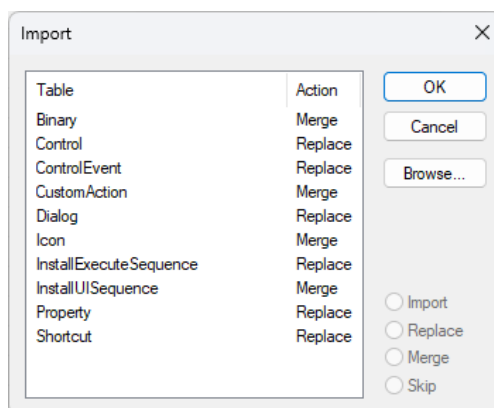
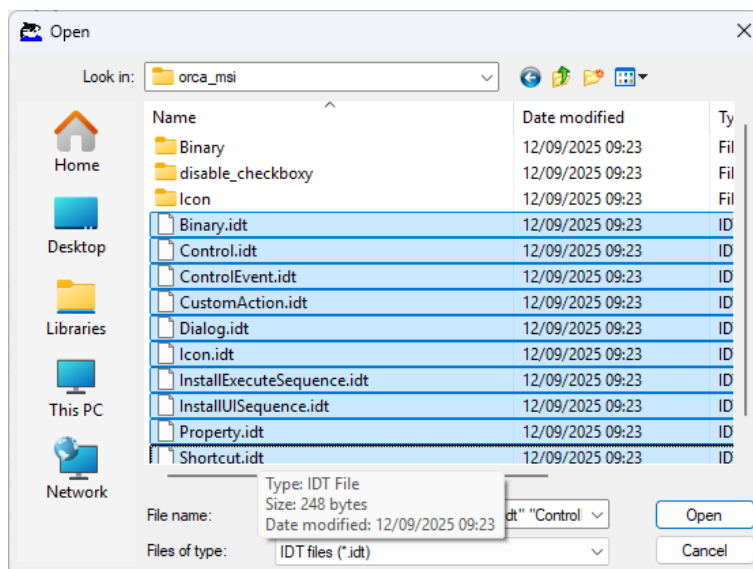
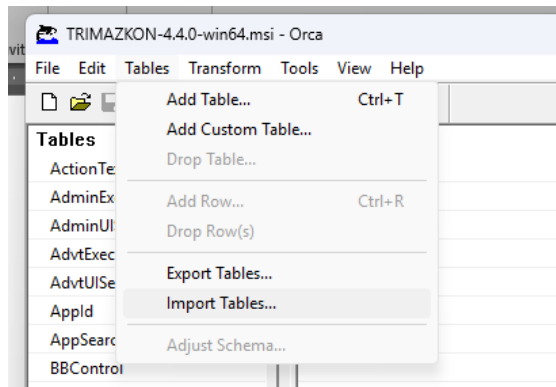
Tento syrový msi installer nainstaluje všechny moduly – budou funkční – ale nebere v potaz nainstalované předešlé verze a mnoho další nekompatibility. Proto je upravován v softwaru Orca, viz níže.

7 Úprava installeru v orca.exe

Pro úpravu installeru je využíván software orca.exe, ke stažení:

<https://learn.microsoft.com/en-us/windows/win32/msi/orca-exe>

- 1) Když v tomto prostředí otevřeme editovaný installer je potřeba udělat několik věcí:
 - a. Import hotových tabulek (nutné mít někde uložené, default: v adresáři dist/orca_msi):



-> musí být vše nastaveno na: „Replace“

b. Dále je potřeba otevřít tabulku „Property“:

Tables	Property	Value
Font	UpgradeCode	{12345678-1234-5678-1234-567812345678}
Icon	INSTALL_OPTION3	0
IniFile	INSTALL_OPTION2	0
IniLocator	INSTALL_OPTION1	1
InstallExecuteSequence	MaintenanceForm_Action	Repair
InstallUISequence	ProductName	TRIMAZKON
IsolatedComponent	Progress2	installs
LaunchCondition	SecureCustomProperties	TARGETDIR;REINSTALLMODE
ListBox	Progress1	Install
ListView	ProductVersion	4.3.5
LockPermissions	DistVersion	4.3.5
MIME	ProductLanguage	1033
Media	CREATE_SHORTCUT	1
MoveFile	RUN_AFTER_INSTALL	1
MsiAssembly	DefaultUIFont	DlgFont8
MsiAssemblyName	PATH_TO_EXE	UNKNOWN
MsiDigitalCertificate	PATH_TO_CMD	C:\Windows\System32\cmd.exe
MsiDigitalSignature	ProductCode	{EEEE5555-FFFF-0435-GGGG-7777HHHH8888}
MsiFileHash	MSIINSTALLPERUSER	1
MsiPatchHeaders	ALLUSERS	2
ODBCAttribute	Manufacturer	JHV - engineering s.r.o
ODBCDataSource	ErrorDialog	ErrorDlg
ODBCDriver	ARPPRODUCTICON	logo_trimazkon
ODBCSourceAttribute		
ODBCTranslator		
Patch		
PatchPackage		
ProgId		
Property		

Nastavit: ProductVersion, DictVersion a ProductCode

V tom ProductCode jen to prostřední číslo podle verze aplikace

c. Nastavit tabulku Component

Tables	Component	ComponentId	Directory_	Attribu...	Condition	KeyPath
CheckBox	TARGETDIR	{5D56E5AA-9A1A-41E4-A9ED-5C7A06436611}	TARGETDIR	256		
Class	convert_application	{9B5B65F-6955-4C12-87FA-CBC85B865CE2}	convert_application	256	INSTALL_OPTION1 = "1"	
ComboBox	images	{5D6DF602-F0EF-4167-B9BD-6A2C676C419D}	images	256		
ComplLocator	_cx_executable2_Executable_script_IP_SET_GUI_v1.py_	{864645FD-7F8B-40CC-AE7B-04215F81C267}	TARGETDIR	256	INSTALL_OPTION2 = "1"	jhv_ip.exe
Complus	_cx_executable1_Executable_script_JHV_MAZ_GUI_v4.py_	{0E7AC748-318D-4798-A570-56B130893F97}	TARGETDIR	256	INSTALL_OPTION3 = "1"	jhv_MAZ.exe
Component	lib	{6DCF79CD-64B7-4E93-8C99-EF8C04564448}	lib	256		
Condition	_cx_executable0_Executable_script_JHV_APP_GUI_v17.py_	{7340DD8F-F698-447C-AED3-BFEC027B10EB}	TARGETDIR	256	INSTALL_OPTION1 = "1"	TRIMAZKON.exe
Control	tcl8.6	{35AF16D0-FB26-4EEC-8C54-F8300551B68C}	tcl8.6	256		

Do kolonky „Condition“ vložit (bez toho v závorce, viz obr.):

INSTALL_OPTION1 = "1" (trimazkon all + convert application)

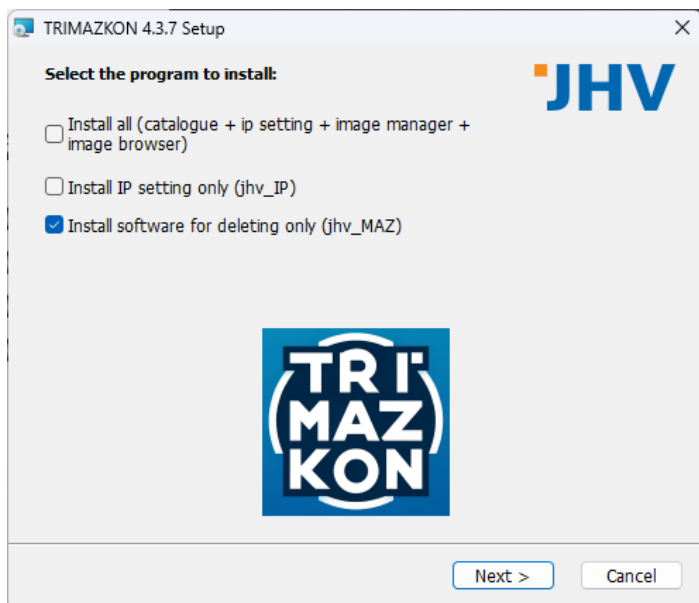
INSTALL_OPTION2 = "1" (jhv_ip)

INSTALL_OPTION3 = "1" (jhv_MAZ)

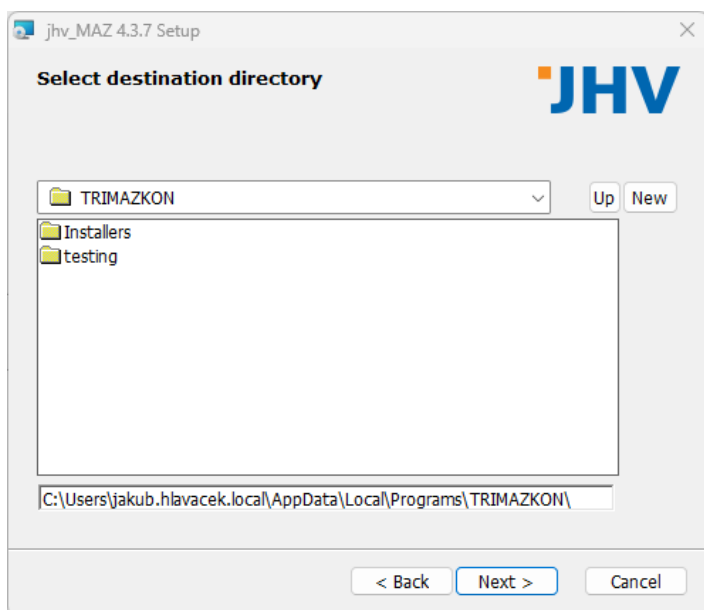
- d. Uložit a hotovo – nyní instalace zjistí, že je nainstalovaná starší verze a odstraní ji a je možné si vybrat modul.

8 Instalace programu

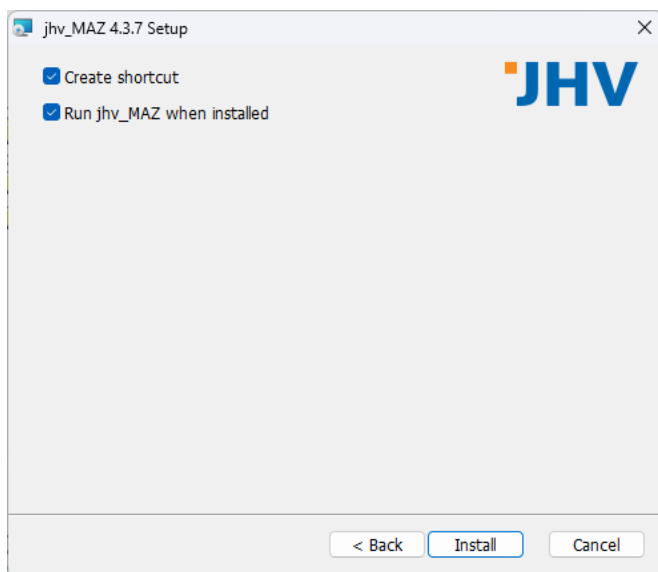
1) Spustíte instalér .msi



Zvolte nástroj pro mazání (jhv_MAZ), next

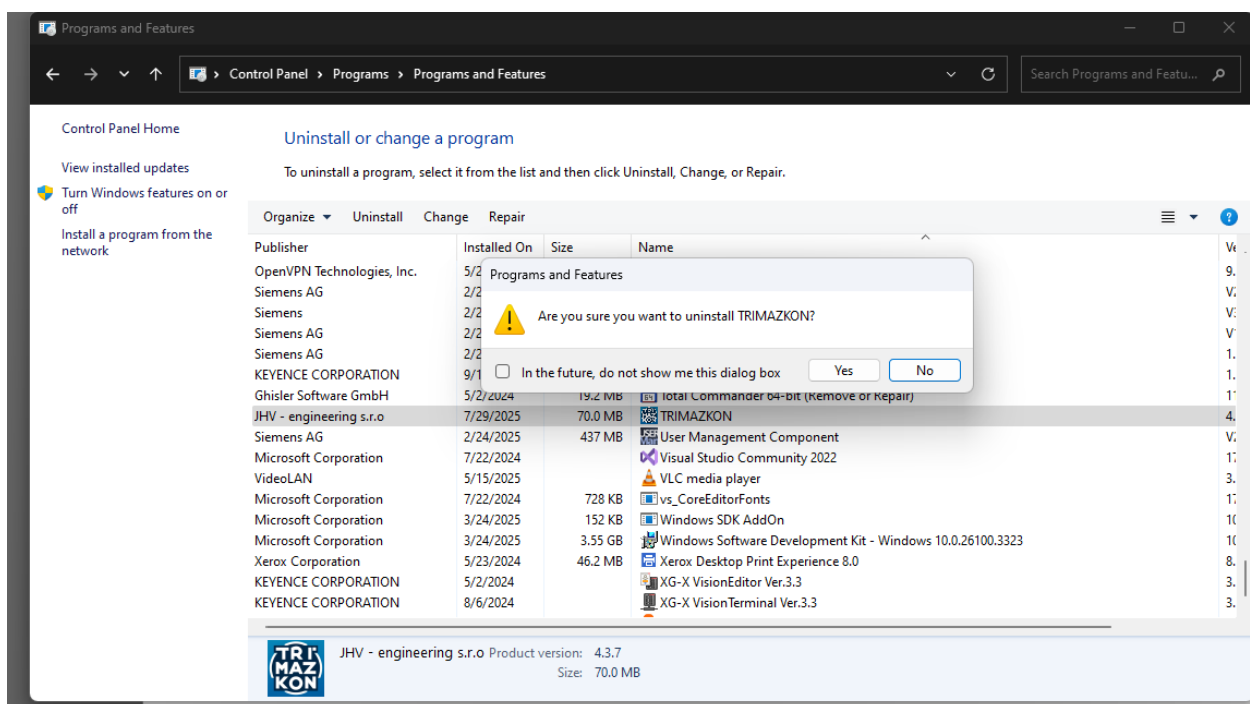


Cestu můžete ponechat defaultní v AppData, next



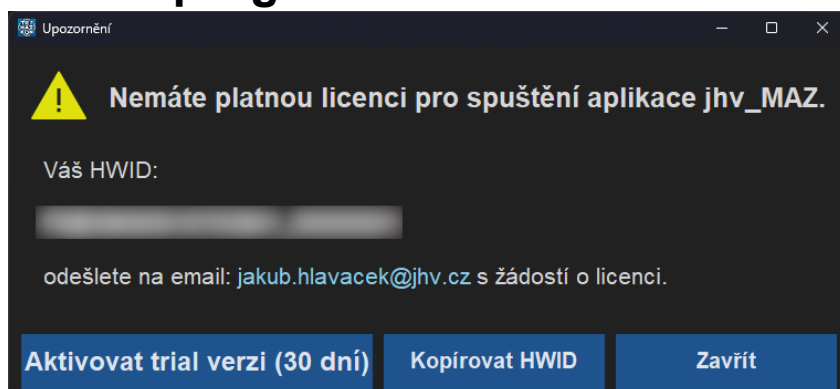
Spustíte instalaci

8.1 Odinstalace programu



Najdete běžně v Control panel a postup je stejný jako u jiných programů

9 Licencování programu



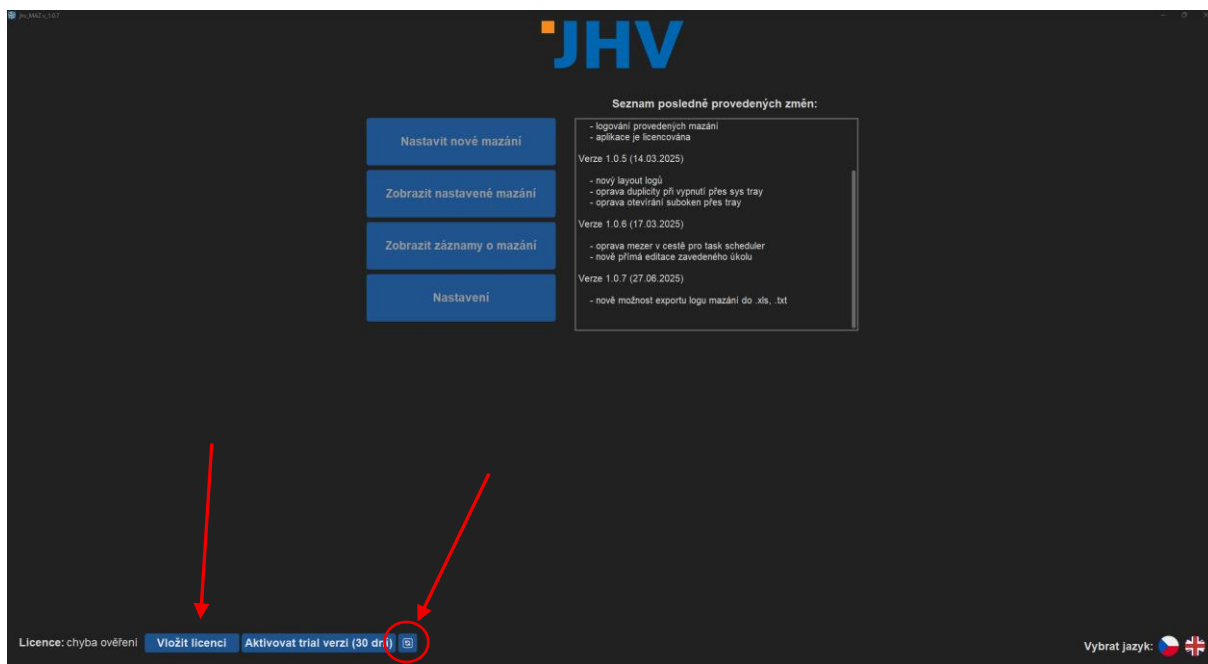
Obrázek 1 Neplatná licence

Krátkodobá licence:

- možnost zkušební verze na 30 dní (tlačítko: aktivovat trial verzi)

Dlouhodobá licence:

- je šifrována na základě HWID zařízení, na kterém je aplikace spouštěna – o dlouhodobou licenci je třeba požádat (zkopírovat HWID a odeslat na uváděnou adresu: jakub.hlavacek@jhv.cz).



Obrázek 2 Vložení dlouhodobé licence

- Po zažádání a obdržení licenčního souboru (license.lic) je potřeba jej přesunout do složky (přes tlačítko „Vložit licenci“ v hlavním menu).
- Pro aktualizaci (po vložení nové licence) slouží tlačítko se dvěma šipkami (Obr. 2, červená šipka vpravo)

9.1 Šifrování nových licencí

Nová licence se generuje pomocí .py souboru `sifrovani_licence.py`. Funguje na principu dvou vygenerovaných rsa klíčů: `private.pem` a `public.pem`. Public klíč je vždy zabalen s installerem aplikace. K vygenerování nové licence potřebujete privátní klíč a hwid zařízení, kde poběží.

```
def make_licence(hwid):
    """
    pro každého uživatele je možné nastavit jeho HW
    - aby se to nešířilo na více zařízení
    """
    # Načtení soukromého klíče
    with open("private.pem", "rb") as f:
        private_key = serialization.load_pem_private_key(
            f.read(), password=None
        )

    # Licence
    licence_data = hwid+"EXPIRES:31.12.9999"

    # Podepsání licence soukromým klíčem
    signature = private_key.sign(
        licence_data.encode(),
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                    hashes.SHA256())
    )

    # Uložení licence a podpisu do souboru
    with open("license.lic", "w") as f:
        f.write(licence_data + "\n")
        f.write(signature.hex())
```

Zde se nastaví expiration date. Pak už stačí funkci akorát zavolat s hwid:

```
# make_licence("0041323K03N0173")
# make_licence("0025_3848_41A1_B7")
# make_licence("E823_8FA6_BF53_00")
# make_licence("E823_8FA6_BF53_00")
# make_licence("ACE4_2E00_314A_D2")
# make_licence("0025_38D6_3140_E5")
make_licence("0000_0000_0000_0000")
```

9.2 Změna kontaktního emailu pro požadavek licence

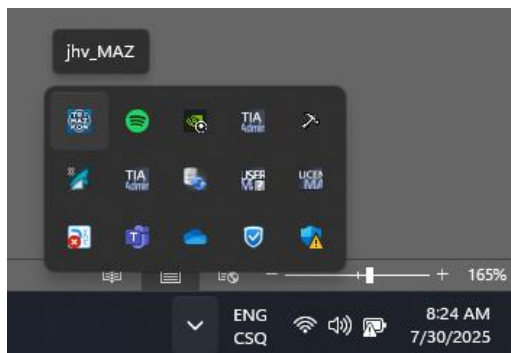
Je tak možné udělat přímo v kódu v horních částech GUI modulů. Nebo vždy u .json konfiguračního souboru, který si aplikace sama vygeneruje po instalaci.

```
{
  "app_settings": {
    "default_path": "C:/Users/",
    "maximalized": "ano",
    "show_changelog": "ne",
    "app_zoom": 130,
    "app_zoom_checkbox": "ano",
    "tray_icon_startup": "ne",
    "default_language": "cz",
    "tooltip_status": "ano",
    "github_repo_owner": "Jakubhl",
    "github_repo": "Work",
    "licence_email_contact": "jakub.hlavacek@jhv.cz"
  },
  "sort_conv_settings": {
    "supported_formats_sorting": [
      "bmp",

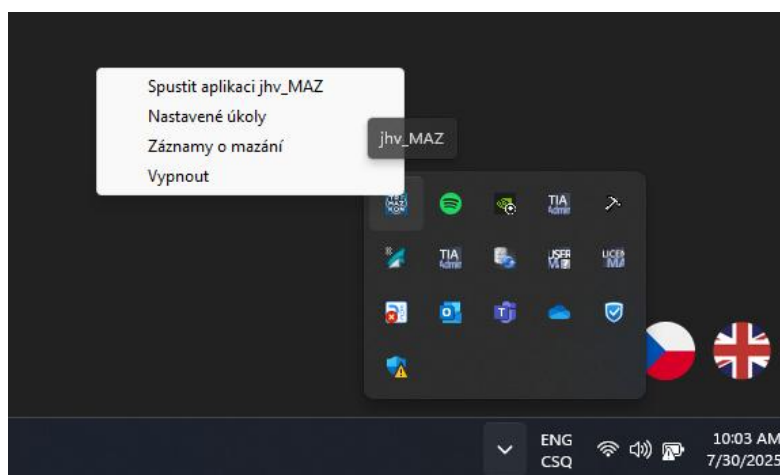
```

10 Ovládání přes systémovou nabídku „tray icons“

Po spuštění systému poté bude aplikace přístupná z nabídky, v pravém dolním rohu, na systémové liště:



Pravým klikem se rozbalí menu:



Obrázek 3 Nabídka v prostředí tray icons

Umožňuje (GUI jhv_MAZ.exe):

- 1) Přímou aplikaci spustit
- 2) Zobrazit nastavené úkoly – otevře průvodce
- 3) Zobrazit záznamy o provedených mazáních – otevře průvodce
- 4) Vypnout aplikaci úplně (navolené, pravidelně vykonávané, úkoly se však budou vykonávat)

V plné verzi aplikace:

- 1) Přímou aplikaci spustit
- 2) Přímou nastavit ip adresy u různých interfaců manuálně
- 3) Nastavit ip adresy různých interfaců na uložené adresy v konfigurační
- 4) Zobrazit nastavené úkoly – otevře průvodce
- 5) Zobrazit záznamy o provedených mazáních – otevře průvodce
- 6) Vypnout aplikaci úplně (navolené, pravidelně vykonávané, úkoly se však budou vykonávat)

11 Seznam souborů potřebných k uchování mimo git

Nešířit veřejně – přidat do gitignore:

- 1) public.pem
- 2) private.pem